

ParamAIttric Local — Lemonade Integration Specification

Status: Initial architecture and implementation plan

Primary objective: Prepare the existing ParamAIttric project to operate reliably with local LLMs through Lemonade while preserving Claude as the reference implementation and creating a practical path toward Windows, Linux, Fusion, and FreeCAD support.

1. Product Definition

ParamAIttric Local is a private maintenance CAD assistant that converts plain-language descriptions and measurements into validated, editable, and printable utility parts.

The initial version will:

- Run a local LLM through Lemonade.
- Use Pi as the agent harness.
- Connect to ParamAIttric through MCP.
- Use Autodesk Fusion as the first production CAD backend.
- Support NVIDIA and AMD inference hardware through selectable Lemonade profiles.
- Preserve the existing Claude workflow as the quality baseline.
- Prepare for a Linux-native FreeCAD backend.
- Export verified STL files and, where supported, editable CAD documents.

The maintenance use case remains central:

- Replacement brackets
- Spacers and bushings
- Covers and guards
- Equipment adapters
- Mounting plates
- Handles and knobs
- Small discontinued or difficult-to-procure utility parts

ParamAIttric already has the appropriate product structure: an AI host selects constrained MCP tools, the server validates requests and manages staged workflows, and the Fusion add-in performs CAD operations.

2. Architectural Principles

2.1 Preserve the working Claude implementation

Claude remains the reference system.

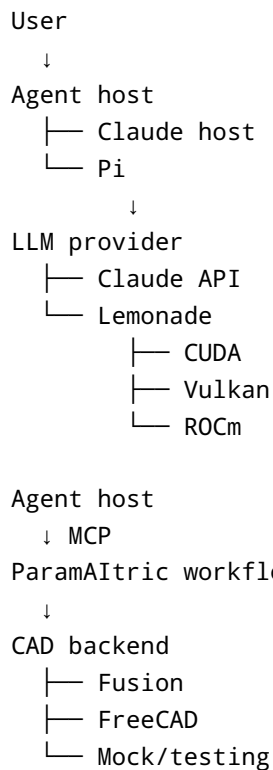
Local-model work must not weaken:

- Existing workflows
- Fusion operation reliability
- Geometry verification
- Structured failures
- STL export
- Guided freeform safeguards

Every major local-model change will be evaluated against the same requests using Claude.

2.2 Separate model hosting from CAD execution

The model provider and CAD backend are independent choices.



Changing the model endpoint must not require changing CAD workflow code.

Changing the CAD backend must not require changing the agent or model.

2.3 Do not fork Pi

Use Pi through its supported extension and model-provider mechanisms.

ParamAItric-specific behavior should live in:

- MCP tools
- ParamAItric prompts
- Runtime profiles
- A small Pi extension only where necessary
- Launcher and diagnostic scripts

2.4 Keep deterministic CAD logic outside the LLM

The model should:

- Understand the request
- Select a workflow
- Gather missing measurements
- Populate validated parameters
- Interpret structured results
- Explain failures

The model should not:

- Generate arbitrary scripts and execute them automatically
- Guess safety-critical dimensions
- Bypass verification
- Continue after failed geometry checks
- Invent unsupported CAD operations

2.5 Do not port brittle assumptions to FreeCAD

ParamAItric's current roadmap correctly identifies semantic selection, stable references, and a narrow operation vocabulary as its most important internal work. It explicitly warns against prematurely expanding to broader backend work before those foundations improve.

The FreeCAD effort will therefore begin as a limited backend-contract spike rather than a broad rewrite.

3. Runtime Profiles

Runtime configuration will be explicit rather than inferred.

Recommended profiles:

```
claude-fusion
lemonade-cuda-fusion
lemonade-vulkan-fusion
lemonade-rocm-fusion-remote
```

```
lemonade-vulkan-freecad  
lemonade-rocm-freecad
```

Example conceptual configuration:

```
{  
  "profile": "lemonade-cuda-fusion",  
  "agent": "pi",  
  "model_provider": "lemonade",  
  "model_endpoint": "http://127.0.0.1:13305/api/v1",  
  "model": "Qwen3.5-9B-GGUF",  
  "inference_backend": "cuda",  
  "cad_backend": "fusion",  
  "cad_endpoint": "http://127.0.0.1:8123",  
  "tool_profile": "guided",  
  "export_directory": "~/Documents/ParamAItric Exports"  
}
```

Profiles belong outside the core workflow implementation.

4. Phase 0 — Establish the Golden Baseline

Goal

Capture the current successful Claude behavior before changing the host interface.

Deliverables

Create an evaluation set of approximately 12–20 maintenance requests covering:

- Spacer
- Simple plate
- Plate with one hole
- Plate with multiple holes
- Cylinder
- Tube
- L-bracket
- Filleted bracket
- Chamfered part
- Box or enclosure
- Unsupported request
- Ambiguous request with missing measurements
- Intentional invalid dimensions
- Fusion bridge unavailable

- Geometry verification failure

Each case records:

- Original user request
- Expected recommended workflow
- Required measurements
- Expected tool call
- Expected normalized arguments
- Expected verification facts
- Expected export type
- Whether the request should succeed or fail safely

Acceptance criteria

- Claude completes all currently supported golden cases.
- Unsupported and invalid requests fail safely.
- Generated files can be located and opened.
- Verification output is recorded.
- Existing test suite remains green.

The baseline becomes the regression contract for local models.

5. Phase 1 — Make ParamAItric Local-Model Ready

This phase improves the existing project for every AI host, not just Lemonade.

5.1 Improve MCP schema fidelity

The current endpoint dynamically registers each ordinary workflow with a function shaped approximately as:

```
def workflow_tool(payload: dict) -> dict:  
    ...
```

That leaves the host-facing schema centered on a generic dictionary instead of exposing the exact structured fields required by each workflow.

Claude can often compensate for broad schemas using descriptions and reasoning. A 9B local model will benefit substantially from explicit field names, types, ranges, units, required values, and enums.

Required improvement

Generate precise MCP input schemas from the existing Pydantic or workflow definitions.

The model should see something conceptually similar to:

```
{
  "width_cm": {
    "type": "number",
    "minimum": 0.1,
    "description": "Overall plate width in centimeters"
  },
  "height_cm": {
    "type": "number",
    "minimum": 0.1
  },
  "thickness_cm": {
    "type": "number",
    "minimum": 0.05
  },
  "hole_diameter_cm": {
    "type": "number",
    "minimum": 0.01
  }
}
```

This is one of the highest-value changes for local-model reliability.

5.2 Provide two tool profiles

Full profile

Designed for:

- Claude
- Development
- Larger models
- Direct testing

It exposes all available, implemented workflow tools with precise schemas.

Guided profile

Designed for:

- Qwen3.5 9B
- Smaller local models
- Contest demonstration
- Novice users

Suggested surface:

```
cad_health
cad_recommend_workflow
cad_get_requirements
cad_build
cad_inspect
```

Typical sequence:

```
recommend workflow
  ↓
retrieve exact requirements
  ↓
gather missing values
  ↓
build validated workflow
  ↓
inspect and report
```

Both tool profiles will call the same underlying workflow implementations.

The evaluation suite will determine whether the 9B model performs better with typed direct tools or the guided facade.

5.3 Normalize structured errors

Every failure returned to a host should contain:

```
{
  "ok": false,
  "classification": "validation_error",
  "stage": "extrude_base",
  "error": "Thickness must be greater than zero.",
  "recoverable": true,
  "next_step": "Ask the user for a positive thickness.",
  "partial_result": {}
}
```

No local model should need to interpret Python exceptions or raw tracebacks.

5.4 Add capability-aware health information

Health should eventually report:

```
{
  "ready": true,
  "cad_backend": "fusion",
  "backend_version": "...",
  "mode": "live",
  "capabilities": [
    "sketch",
    "extrude",
    "cut",
    "fillet",
    "chamfer",
    "stl_export"
  ],
  "workflow_count": 24
}
```

Prompts should no longer hardcode “Fusion” where the active backend can provide the answer.

5.5 Add a diagnostic command

Introduce a command such as:

```
paramaitric doctor --profile lemonade-cuda-fusion
```

It should test:

- Python environment
- ParamAItric package import
- MCP startup
- Lemonade endpoint
- Requested model availability
- CAD backend reachability
- Bridge authorization
- Export-directory write permission
- One non-mutating health call

5.6 Complete relevant reliability work

Before public packaging:

- Finish per-run bridge authorization.
- Add browser-origin protections.
- Define server-side dispatch deadlines.
- Define late-mutation cancellation policy.
- Keep package clean-install CI working.

- Add the intended open-source license.
- Preserve truthful workflow availability.

Most packaging and workflow-availability cleanup is already complete, while authorization, dispatch deadlines, and license selection remain open.

6. Phase 2 — NVIDIA Laptop Integration

Configuration

```
Windows laptop
├─ Fusion
├─ ParamAItrix add-in
├─ ParamAItrix MCP server
├─ Pi
└─ Lemonade
```

Backend order

1. **CUDA baseline**
2. **Vulkan portability test**
3. CPU only as a diagnostic fallback

Lemonade supports Vulkan across AMD, NVIDIA, and Intel GPUs, but its backend guidance recommends CUDA first on modern NVIDIA hardware because CUDA will typically outperform Vulkan.

Therefore:

- Do not design ParamAItrix around CUDA.
- Do use CUDA for the first reliable laptop baseline.
- Run the identical tests using Vulkan afterward.
- Store the backend only in the runtime profile.

Model order

1. Qwen3.5 9B
2. gpt-oss 20B
3. Optional smaller model as a lower-bound experiment

Measurements collected

For every golden request:

- Correct workflow selected
- Correct tool selected

- Valid JSON arguments
- Number of retries
- Number of unnecessary questions
- Hallucinated parameters
- CAD execution result
- Verification result
- Export result
- Time to first tool call
- Total completion time
- Prompt and completion token counts where available

Acceptance criteria for the 9B model

- Completes at least 10 of the first 12 supported golden requests without manual tool correction.
- Never bypasses a failed verification.
- Never invents unavailable operations.
- Fails safely on invalid and unsupported cases.
- Produces the same final verified geometry as the Claude baseline for successful cases.

The exact pass threshold can be revised after the first baseline run, but safety failures remain zero-tolerance.

7. Phase 3 — FreeCAD Feasibility Spike on the Laptop

Purpose

Determine whether ParamAItrix can support a Linux-native, completely offline CAD path without committing to a full port.

FreeCAD is an open-source parametric modeler that runs on Windows, Linux, and macOS. It uses OpenCascade and provides a broad Python API.

Expected effort

The high-level workflows should be reusable.

The low-level CAD implementation will not be drop-in compatible because Fusion and FreeCAD use different:

- Object models
- Sketch systems
- Feature histories
- Persistent references
- Selection mechanisms
- Threading and GUI APIs
- Document formats

This will probably be a **moderate port of the operation layer**, not a minor configuration change.

Spike architecture

Implement a FreeCAD backend that speaks the same external bridge contract:

```
GET /health
POST /command
POST /cancel
GET /capabilities
```

Initial supported commands:

```
create_document
create_sketch
draw_rectangle
draw_circle
extrude_profile
cut_profile
inspect_body
export_stl
export_step
save_native_document
```

First workflow

Port exactly one workflow first:

Plate with centered hole

Required outputs:

- Valid FreeCAD document
- Valid solid
- `.FCStd`
- `.STEP`
- `.STL`
- Bounding-box measurements
- Volume
- Body count
- Hole verification

Second workflow, only after success

Simple L-bracket

This adds:

- Multiple modeling stages
- More complex placement
- Edge or face selection
- Fillet or hole placement

FreeCAD spike acceptance criteria

Proceed when:

- The existing ParamAItric workflow can run without backend-specific branches throughout the workflow.
- Backend-specific code remains behind the backend interface.
- The FreeCAD result matches the expected dimensions.
- The native FreeCAD document opens and remains editable.
- STEP and STL exports pass validation.
- Failures return the same structured error format as Fusion.

Stop conditions

Stop the FreeCAD expansion and document findings when:

- Shared workflows require extensive `if fusion / if freecad` branching.
- One basic workflow requires rewriting the MCP or agent layers.
- Stable geometry identification cannot be expressed through the planned semantic selector vocabulary.
- The backend cannot produce dependable native documents.
- The spike begins interfering with current Fusion reliability work.

A stopped spike is still valuable because it establishes the true porting boundary.

8. Phase 4 — Strix Halo Deployment

Stage 4A: Remote AMD inference

Keep CAD on the laptop.

```
Windows laptop
├─ Fusion
├─ ParamAItric
└─ Pi
    └─ ↓ local network or SSH tunnel
```

```
Strix Halo Ubuntu
└─ Lemonade + local model
```

Backend order on Strix Halo:

1. ROCm
2. Vulkan
3. CPU diagnostic fallback

Lemonade specifically supports Ryzen AI Strix Point and Strix Halo through its ROCm backend and recommends ROCm first, with Vulkan as fallback.

Run the same golden evaluation set used on the laptop.

This isolates hardware inference testing from CAD migration.

Stage 4B: Native Linux offline stack

```
Strix Halo Ubuntu
├─ Lemonade
├─ Pi
├─ ParamAItrix
└─ FreeCAD backend
```

This is the target configuration for the strongest offline claim.

Acceptance criteria:

- No cloud model connection required.
- No Autodesk application required.
- No Windows host required.
- Native CAD document plus STEP and STL output.
- Same evaluation and verification contract as the laptop version.

Stage 4C: Fusion-on-Linux experiments

Try these only after the native Lemonade and FreeCAD paths are understood.

Wine

Treat as experimental.

A community-maintained Wine installation path exists, but it explicitly notes that Linux distributions are not officially supported by Autodesk.

Test:

- Fusion startup
- Authentication
- Graphics stability
- Add-in loading
- Loopback bridge
- Main-thread custom events
- File export
- Behavior after Fusion updates

Failure here does not block the project.

Windows virtual machine

Treat as a last-resort research path.

A VM introduces:

- Windows installation and licensing
- GPU acceleration or passthrough
- Shared-folder complexity
- USB and graphics integration
- Additional memory consumption
- More difficult packaging

For an integrated-GPU Strix system, native Windows or dual boot is likely a cleaner Fusion test environment than building the product around a VM.

9. Phase 5 — Packaging

Development packaging

Initially package:

- ParamAItrix Python environment
- Pi integration
- Runtime profiles
- Prompt files
- Diagnostic command
- Model setup instructions
- Fusion add-in installer
- FreeCAD backend when viable
- Evaluation suite

Product packaging

Later transition toward:

- Embeddable Lemonade
- Guided model download
- One-command startup
- Automatic hardware/backend detection
- Windows installer
- Linux package or AppImage
- Offline documentation
- Model and CAD health dashboard
- Export history
- Optional Fusion or FreeCAD backend selection

Lemonade explicitly offers both a standalone server and an embeddable form intended for packaging local AI into applications, so beginning with Lemonade Server does not close off a more integrated product later.

10. Recommended Repository Structure

```
paramAItric/  
├─ mcp_server/  
├─ fusion_addin/  
├─ cad_backends/  
│   ├── protocol.py  
│   ├── fusion/  
│   ├── freecad/  
│   └─ mock/  
├─ local_app/  
│   ├── profiles/  
│   │   ├── claude-fusion.json  
│   │   ├── lemonade-cuda-fusion.json  
│   │   ├── lemonade-vulkan-fusion.json  
│   │   ├── lemonade-rocm-remote.json  
│   │   └─ lemonade-rocm-freecad.json  
│   ├── prompts/  
│   ├── pi_extension/  
│   └─ doctor/  
├─ evaluations/  
│   ├── cases/  
│   ├── expected/  
│   ├── results/  
│   └─ runner/  
├─ scripts/  
└─ start_windows.ps1
```

```

|   |— start_linux.sh
|   |— connect_strix.ps1
|   |— run_evaluations.py
|— docs/
|   |— LEMONADE_INTEGRATION.md
|   |— LOCAL_MODEL_EVALUATION.md
|   |— CAD_BACKEND_PROTOCOL.md
|   |— FREECAD_SPIKE.md

```

Do not move existing Fusion files merely to achieve this appearance. Introduce directories incrementally as working implementations emerge.

11. Immediate Implementation Order

First implementation batch

1. Create the Lemonade integration branch.
2. Add the golden Claude evaluation cases.
3. Add runtime-profile parsing.
4. Add the diagnostic command skeleton.
5. Improve MCP input schema fidelity.
6. Preserve a full tool profile.
7. Add the guided local-model tool profile.
8. Install and validate Lemonade with CUDA on the laptop.
9. Connect Pi to ParamAItrix.
10. Complete one golden Fusion workflow with Qwen3.5 9B.
11. Run the same workflow through Vulkan.
12. Expand to the remaining golden cases.
13. Test gpt-oss 20B after the 9B results are recorded.

Second implementation batch

1. Finish bridge authorization and timeout behavior.
2. Continue attribute pinning and stable-reference work.
3. Define the minimal backend operation vocabulary.
4. Write the CAD backend protocol.
5. Perform the one-workflow FreeCAD spike.
6. Decide whether the FreeCAD backend proceeds or pauses.

Third implementation batch

1. Install Lemonade on Strix Halo.
2. Run the model remotely over ROCm.
3. Repeat through Vulkan.
4. Compare laptop and Strix evaluation results.
5. Move the FreeCAD backend to Ubuntu if the laptop spike passed.

6. Test Wine only after the native Linux path is working.

12. Roadmap Decision Summary

The recommended sequence is:

```
Current Claude + Fusion baseline
      ↓
Local-model readiness improvements
      ↓
Laptop: Lemonade + CUDA + Fusion
      ↓
Laptop: Lemonade + Vulkan + Fusion
      ↓
Laptop: limited FreeCAD spike
      ↓
Strix: remote ROCm/Vulkan inference
      ↓
Strix: native Lemonade + FreeCAD
      ↓
Experimental Fusion through Wine
      ↓
Optional native Windows or VM investigation
```

This sequence produces useful deliverables at every step and prevents the contest integration from becoming disposable work.

The central product architecture becomes:

One verified ParamAItric workflow core, multiple model providers, multiple inference backends, and eventually multiple CAD backends.